

UNIVERSIDADE FEDERAL DE MINAS GERAIS
Programa de Graduação em Engenharia de Sistemas
Departamento de Ciências da Computação
Introdução a Banco de Dados

Samuel Lima Horta
2023060561
Thiago Tobias Valente de Oliveira Santos
2023060790
Bernardo Soares Diniz Lara
2023060898
Luigi Pinesi Tavares Jacob
2023079939

Trabalho Prático -
Modelagem, Implementação e Análise de uma Base de Dados
Aberta

Belo Horizonte
17 de Junho 2026

Modelagem, Implementação e Análise de uma Base de Dados Aberta

1 Dataset escolhido

O dataset escolhido para o nosso trabalho, e previamente aprovado pelo professor, é composto por uma base de dados que reúne informações relacionadas a competições de futebol, contendo dados sobre partidas, times e jogadores. Para este projeto, foi utilizada a tabela referente ao Campeonato Brasileiro Série A, abrangendo o período de 2003 a 2024, contendo diversos detalhes sobre as partidas e outras informações relevantes de cada temporada.

Abaixo, deixamos anexados os links para acesso aos dados utilizados no trabalho, tanto na plataforma Base dos Dados quanto no repositório do GitHub desenvolvido para o projeto.

Link da Base dos Dados:

<https://basedosdados.org/dataset/c861330e-bca2-474d-9073-bc70744a1b23?table=18835b0d-233e-4857-b454->

Link do repositório GitHub: https://github.com/samucao1803/TrabalhoPratico_IBD_BrasileiraoDBAnalysis

2 Projeto Conceitual - Modelo ER/EER

Dado o dataset escolhido, foi necessário realizar um processo de modelagem com o objetivo de projetar um modelo conceitual capaz de representar adequadamente a realidade dos dados. Nessa etapa inicial, foi possível identificar entidades, atributos e possíveis chaves de forma preliminar. As entidades e os atributos identificados encontram-se listados a seguir.

- **Entidade:** Campeonato
Atributos: id_campeonato (PK), ano_campeonato.
- **Entidade:** Partida
Atributos: id_partida (PK), data, rodada, publico, gols_mandante, gols_visitante.
- **Entidade:** Time
Atributos: id_time (PK), nome_time.
- **Entidade:** Pessoa (Superclasse)
Atributos: id_pessoa (PK), nome.
- **Entidade:** Técnico (Subclasse de Pessoa)
Atributos: id_tecnico (PK).
- **Entidade:** Árbitro (Subclasse de Pessoa)
Atributos: id_arbitro (PK).
- **Entidade:** Estádio
Atributos: id_estadio (PK), nome_estadio, publico_max.
- **Entidade:** Estatística
Atributos: id_estatistica (PK), tipo_mando, colocacao, valor_equipe_titular, idade_media_titular, escanteios, faltas, chutes_bola_parada, defesas, impedimentos, chutes, chutes_fora.

Em uma segunda análise, com base nos dados presentes na tabela e na etapa anterior de identificação das entidades, também foi possível identificar possíveis relacionamentos entre elas. Esses relacionamentos foram refinados durante o processo de modelagem, permitindo a definição de suas respectivas cardinalidades, de acordo com a natureza das entidades envolvidas e com a forma como elas se relacionam no contexto do mundo real:

- **Relacionamento: Campeonato – Partida**
Cardinalidade: Campeonato (1) → (N) Partida.
- **Relacionamento: Estádio – Partida**
Cardinalidade: Estádio (1) → (N) Partida.

- **Relacionamento: Árbitro – Partida**
Cardinalidade: Árbitro (1) $\rightarrow$ (N) Partida.
- **Relacionamento: Partida – Estatística**
Cardinalidade: Partida (1) $\rightarrow$ (N) Estatística.
- **Relacionamento: Time – Estatística**
Cardinalidade: Time (1) $\rightarrow$ (N) Estatística.
- **Relacionamento: Técnico – Estatística**
Cardinalidade: Técnico (1) $\rightarrow$ (N) Estatística.
- **Relacionamento: Pessoa – Técnico / Árbitro (Especialização/Generalização)**

Por fim, com a evolução da modelagem, foi possível estabelecer algumas restrições de integridade fundamentais para garantir a consistência, a confiabilidade e a estrutura adequada dos dados. Essas restrições têm como objetivo preservar a integridade do modelo e evitar inconsistências durante sua utilização. A seguir, são apresentadas as três principais classes de restrições de integridade identificadas pelo grupo:

- **Restrições de Domínio:**

ano_campeonato deve ser maior que zero; rodada deve ser maior que zero; publico deve ser maior ou igual a zero; publico_max deve ser maior que zero; gols_mandante e gols_visitante devem ser maiores ou iguais a zero; colocacao deve estar entre 1 e o número máximo de equipes participantes; valor_equipe_titular deve ser maior ou igual a zero; idade_media_titular deve ser maior que zero; escanteios, faltas, chutes_bola_parada, defesas, impedimentos, chutes e chutes_fora devem ser maiores ou iguais a zero.

- **Restrições de Entidade:**

id_campeonato deve ser único; id_partida deve ser único; id_time deve ser único; id_tecnico deve ser único; id_arbitro deve ser único; id_estadio deve ser único; id_estatistica deve ser único.

- **Restrições de Participação:**

A participação da entidade Partida é total nos relacionamentos com Campeonato, Estádio e Árbitro. A participação de Árbitro, Técnico e Time é parcial em relacionamentos. A participação entre Partida e Estatística é total para ambas. A participação da superclasse Pessoa é total na especialização.

Com a conclusão das etapas de identificação de entidades, relacionamentos e restrições de integridade, foi possível finalizar a modelagem conceitual do projeto, estabelecendo as bases para o seu desenvolvimento a partir do dataset analisado. A seguir, são apresentadas as representações dos dados modelados em forma de diagramas, utilizando a notação ER clássica.

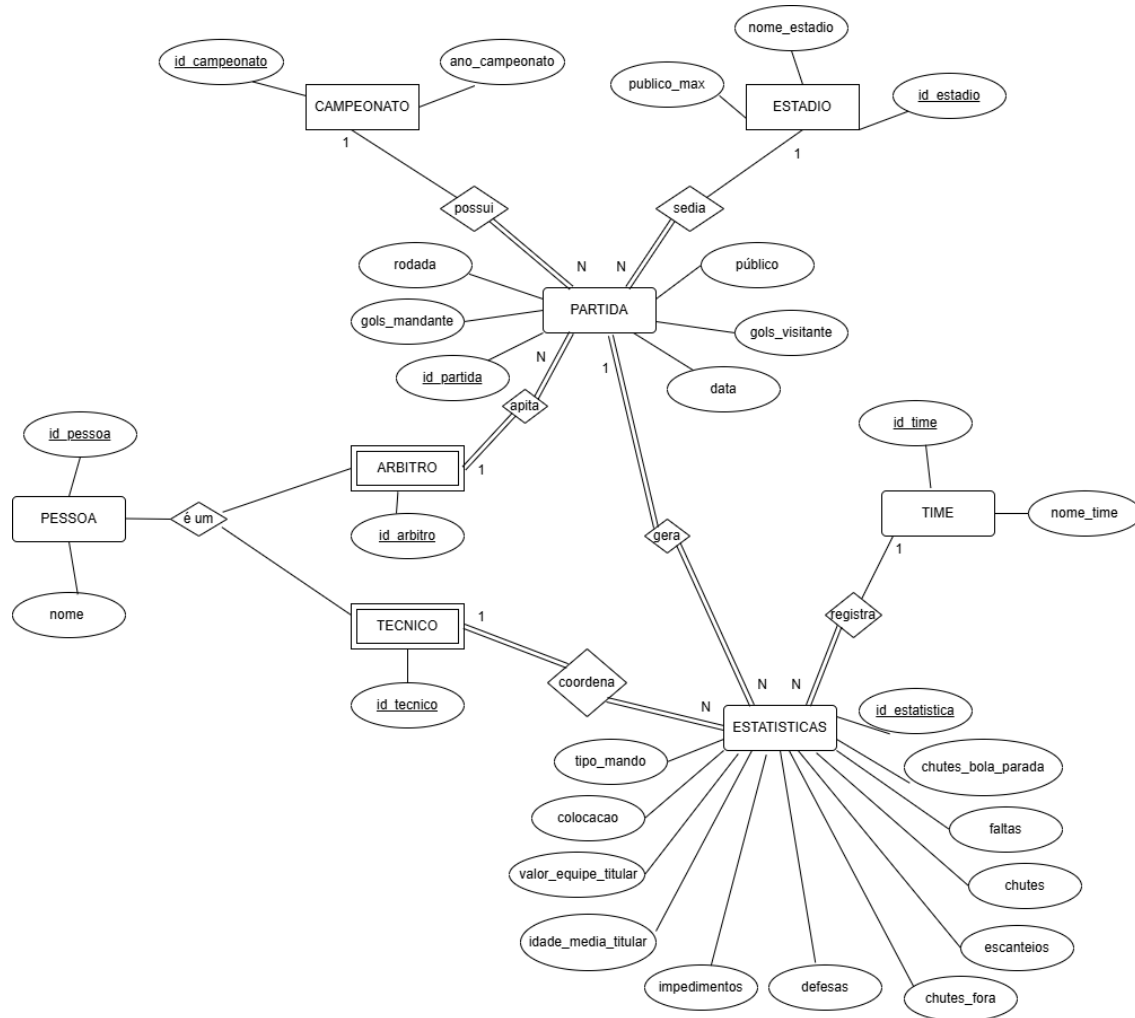


Figura 1: Diagrama de Entidade-Relacionamento do projeto.

3 Modelo Relacional e Normalização

Com base no modelo ER/EER apresentado na seção anterior, foi realizado o mapeamento para o modelo relacional, preservando as cardinalidades e as restrições de integridade identificadas. Nesta etapa, cada entidade forte foi convertida em uma relação, os relacionamentos 1:N foram representados por chaves estrangeiras no lado N, e a especialização Pessoa → Técnico/Árbitro foi implementada por herança com chave compartilhada.

O esquema relacional proposto é apresentado a seguir:

- **Campeonato**(id_campeonato, ano_campeonato)
- **Pessoa**(id_pessoa, nome)
- **Tecnico**(id_tecnico (FK))
- **Arbitro**(id_arbitro (FK))
- **Estadio**(id_estadio, nome_estadio, publico_max)
- **Time**(id_time, nome_time)

- **Partida**(id_partida, data, rodada, publico, gols_mandante, gols_visitante, id_campeonato (FK), id_estadio (FK), id_arbitro (FK))
- **Estatistica**(id_estatistica, tipo_mando, colocacao, valor_equipe_titular, idade_media_titular, escanteios, faltas, chutes_bola_parada, defesas, impedimentos, chutes, chutes_fora, id_partida (FK), id_time (FK), id_tecnico (FK))

As principais chaves estrangeiras definidas no mapeamento são:

- Partida.id_campeonato → Campeonato.id_campeonato
- Partida.id_estadio → Estadio.id_estadio
- Partida.id_arbitro → Arbitro.id_arbitro
- Estatistica.id_partida → Partida.id_partida
- Estatistica.id_time → Time.id_time
- Estatistica.id_tecnico → Tecnico.id_tecnico
- Tecnico.id_tecnico → Pessoa.id_pessoa
- Arbitro.id_arbitro → Pessoa.id_pessoa

Além da chave primária artificial em Estatistica (id_estatistica), pode-se adotar a restrição de unicidade composta (id_partida, id_time, tipo_mando), garantindo no máximo um registro estatístico por time e tipo de mando em cada partida.

Em relação às dependências funcionais, foram consideradas as seguintes dependências principais:

- id_campeonato → ano_campeonato
- id_pessoa → nome
- id_estadio → nome_estadio, publico_max
- id_time → nome_time
- id_partida → data, rodada, publico, gols_mandante, gols_visitante, id_campeonato, id_estadio, id_arbitro
- id_estatistica → tipo_mando, colocacao, valor_equipe_titular, idade_media_titular, escanteios, faltas, chutes_bola_parada, defesas, impedimentos, chutes, chutes_fora, id_partida, id_time, id_tecnico

A normalização foi analisada até a Terceira Forma Normal (3FN):

- **1FN**: Todas as relações possuem atributos atômicos, sem grupos repetitivos.
- **2FN**: Nas relações com chave simples, todos os atributos não-chave dependem totalmente da chave primária.
- **3FN**: Não foram identificadas dependências transitivas entre atributos não-chave dentro de uma mesma relação. Informações descritivas de entidades distintas (por exemplo, nome de estádio, nome de time e nome de pessoa) foram mantidas em relações próprias, reduzindo redundância e evitando anomalias de atualização.

Com isso, o esquema relacional final atende ao requisito mínimo de normalização solicitado no trabalho (pelo menos 3FN), mantendo aderência ao modelo conceitual da Etapa 1 e deixando a base pronta para a implementação em SQL (DDL e carga) na próxima etapa.

4 Implementação

4.1 Diagrama Relacional

A Figura 2 apresenta o diagrama relacional resultante do modelo conceitual definido nas etapas anteriores.

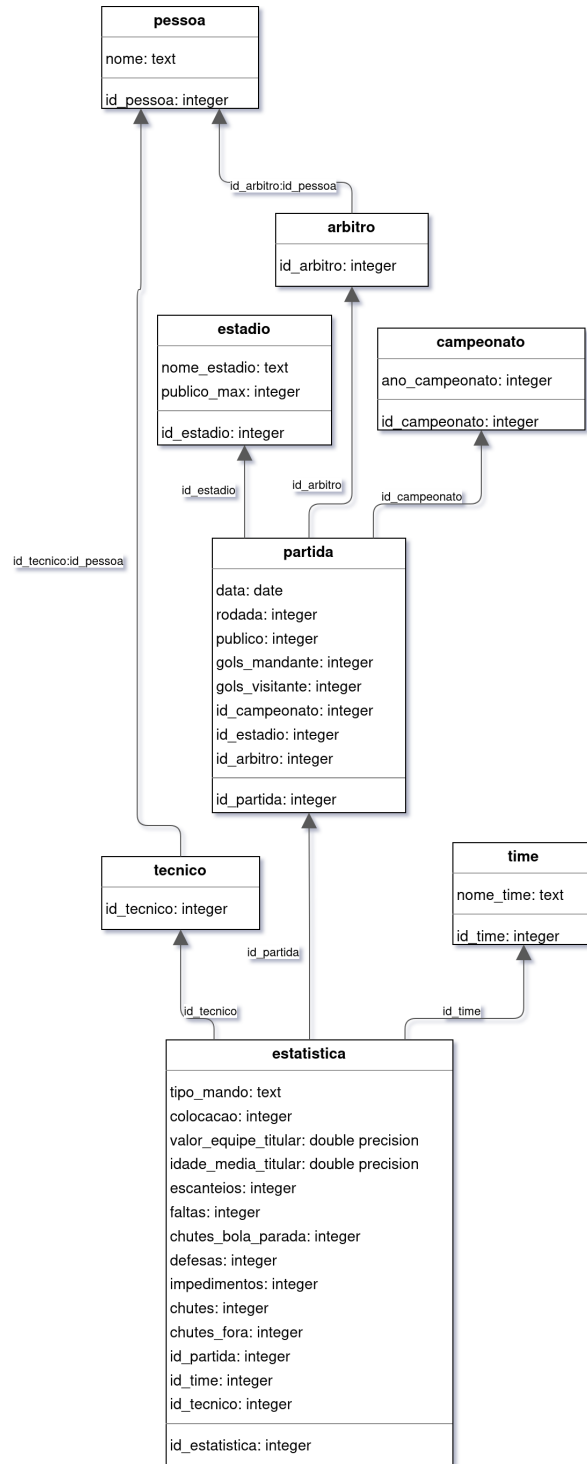


Figura 2: Diagrama relacional do banco de dados

4.2 Tecnologias utilizadas

Para implementação do processo de transformação e carga dos dados foi utilizado um conjunto de ferramentas com funções complementares.

4.2.1 PostgreSQL

O PostgreSQL foi utilizado como Sistema Gerenciador de Banco de Dados (SGBD) responsável pelo armazenamento persistente dos dados processados.

A escolha foi motivada pela robustez, suporte completo ao padrão SQL, tipagem estrita e recursos avançados como *window functions*, CTEs e restrições de integridade expressivas. Ao contrário de soluções embarcadas, o PostgreSQL opera como um servidor dedicado, o que permite conexões simultâneas e é mais representativo de ambientes de produção.

4.2.2 Docker

O Docker foi utilizado para provisionar e isolar o servidor PostgreSQL em um contêiner reproduzível.

O arquivo `docker-compose.yml` define a imagem, as variáveis de ambiente, o mapeamento de porta e o volume persistente dos dados. Essa abordagem elimina a necessidade de instalar o PostgreSQL diretamente no sistema operacional e garante que qualquer colaborador consiga reproduzir o ambiente com um único comando.

4.2.3 Python

Python foi utilizado como linguagem principal para implementação do processo ETL (Extract, Transform and Load).

Sua utilização permitiu automatizar a leitura da base original, realizar as transformações necessárias e executar a população das tabelas relacionais de forma reproduzível e controlada.

4.2.4 Pandas

A biblioteca Pandas foi utilizada para manipulação dos dados em memória.

Seu uso permitiu realizar operações como leitura do arquivo CSV, conversão de tipos, tratamento de valores ausentes, remoção de registros duplicados e preparação dos dados para inserção no banco relacional.

4.2.5 psycopg2

A biblioteca `psycopg2` foi utilizada como adaptador entre Python e PostgreSQL, substituindo o módulo `sqlite3` da implementação anterior.

Uma particularidade relevante é que o PostgreSQL não reconhece nativamente os tipos numéricos do NumPy (`numpy.int64`, `numpy.float64`), que são produzidos pelo Pandas. Por isso, foram registrados adaptadores de tipo explícitos para converter esses valores antes de cada inserção, evitando erros em tempo de execução.

4.2.6 python-dotenv

A biblioteca `python-dotenv` foi utilizada para carregar as credenciais de conexão a partir de um arquivo `.env`, evitando que informações sensíveis como senha do banco fossem incluídas diretamente no código-fonte ou versionadas no repositório.

4.2.7 Visual Studio Code

O Visual Studio Code foi utilizado como ambiente de desenvolvimento para implementação e execução dos scripts.

Sua escolha ocorreu devido ao suporte à linguagem Python, integração com extensões para análise de dados e facilidade de organização do projeto.

4.3 Estrutura do projeto

O projeto foi organizado em artefatos independentes para separar responsabilidades entre configuração do ambiente, armazenamento dos dados e processamento.

A estrutura utilizada foi:

```
Banco de Dados/  
  dados/  
    brasileiro.csv  
    docker-compose.yml  
    .env  
    requirements.txt  
    criar_banco.py
```

4.3.1 Base de dados bruta (CSV)

O arquivo `brasileirao.csv` representa a fonte original dos dados.

Esse arquivo foi mantido sem modificações diretas para preservar reprodutibilidade e permitir repetir o processo de carga sempre que necessário.

4.3.2 Configuração do contêiner (docker-compose.yml)

O arquivo `docker-compose.yml` define o serviço PostgreSQL com suas configurações de ambiente, porta exposta e volume persistente. A persistência é garantida por um volume nomeado, de modo que os dados sobrevivem ao ciclo de vida do contêiner sem necessidade de reexecução do script ETL.

4.3.3 Variáveis de ambiente (.env)

O arquivo `.env` armazena as credenciais de conexão ao banco (host, porta, nome do banco, usuário e senha). Esse arquivo não é versionado no repositório; o arquivo `.env.example` serve como modelo público sem valores sensíveis.

4.3.4 Dependências (requirements.txt)

O arquivo `requirements.txt` lista todas as bibliotecas Python necessárias para execução do projeto, permitindo instalação reproduzível via `pip install -r requirements.txt`.

4.3.5 Script de ETL (criar_banco.py)

O arquivo `criar_banco.py` concentra toda a lógica de processamento dos dados.

Entre suas responsabilidades estão:

- leitura do CSV;
- conversão de tipos;
- tratamento de dados ausentes;
- remoção de duplicatas;
- aplicação das restrições de integridade;
- geração e recriação das tabelas relacionais no PostgreSQL;
- inserção ordenada dos registros;
- validação da carga.

Esse componente representa o núcleo do processo de transformação dos dados. Como o script executa `DROP TABLE IF EXISTS ... CASCADE` antes das criações, cada execução parte de um banco limpo e o processo é totalmente idempotente.

4.4 Processo de carga dos dados (ETL)

Após a definição do modelo relacional, foi desenvolvido um processo de ETL (Extract, Transform and Load) com o objetivo de transformar os dados brutos disponibilizados em formato CSV em um banco relacional estruturado e consistente.

O processo foi dividido em três etapas principais.

4.4.1 Extração

A extração consistiu na leitura do arquivo `brasileirao.csv` utilizando a biblioteca Pandas. O arquivo foi carregado integralmente em memória para permitir as etapas subsequentes de transformação e validação.

4.4.2 Transformação

Após a leitura do arquivo, foi realizada uma etapa de transformação para adequação ao modelo relacional definido anteriormente.

As transformações incluíram:

- Conversão automática de tipos de dados;
- Padronização de valores vazios;
- Tratamento de valores ausentes;
- Remoção de registros duplicados;
- Aplicação das restrições de domínio definidas no modelo conceitual;
- Geração de identificadores artificiais para entidades que não possuíam chave explícita na base original.

Também foi realizada a decomposição das informações da tabela original em múltiplas entidades relacionais.

Cada registro original da base passou a gerar registros distribuídos entre as tabelas Campeonato, Pessoa, Técnico, Árbitro, Estádio, Time, Partida e Estatística.

4.4.3 Carga

Após a validação e transformação dos dados, foi realizada a carga para o PostgreSQL via biblioteca `psycopg2`.

A inserção ocorreu de forma ordenada para respeitar dependências de chaves estrangeiras e garantir integridade referencial.

O fluxo completo da carga pode ser representado por:

CSV → Pandas → Limpeza → Transformação → PostgreSQL

4.5 Tratamento e limpeza dos dados

Durante a etapa de transformação foram aplicadas regras para adequação da base ao modelo relacional e às restrições definidas no projeto.

4.5.1 Conversão de tipos

Os atributos foram convertidos para tipos compatíveis com o esquema relacional do PostgreSQL.

Exemplos:

- `data` → DATE
- `publico` → INTEGER
- `valor_equipe_titular` → DOUBLE PRECISION
- `idade_media_titular` → DOUBLE PRECISION

4.5.2 Tratamento de valores ausentes

Foi adotada uma política distinta para cada categoria de atributo.

- **Atributos obrigatórios** (campeonato, data, rodada, gols, times, estádio e árbitro): registros inválidos foram descartados;
- **Atributos estatísticos opcionais** (escanteios, faltas, chutes, defesas): valores ausentes foram mantidos como NULL;
- **Campos de identificação obrigatória** foram preenchidos quando necessário para preservar integridade referencial.

4.5.3 Remoção de duplicatas

A base passou por remoção de registros repetidos em dois níveis:

- remoção de linhas completamente duplicadas;
- remoção de partidas repetidas considerando: `ano_campeonato + data + rodada + mandante + visitante`.

4.5.4 Aplicação das restrições de domínio

Foram descartados registros que violavam as regras estabelecidas pelo modelo.

Entre elas:

- `ano_campeonato > 0`;
- `rodada > 0`;
- `publico ≥ 0`;
- `gols ≥ 0`;
- `valor_equipe_titular ≥ 0`;
- `idade_media_titular > 0`;
- `estatísticas esportivas ≥ 0`.

4.5.5 Geração de identificadores

Como parte da base original não possuía identificadores compatíveis com o modelo relacional proposto, foram geradas chaves artificiais para as entidades Campeonato, Estádio, Time e Pessoa. As entidades Técnico e Árbitro reutilizaram os identificadores da superclasse Pessoa conforme o modelo de especialização adotado.

4.5.6 Resultado da limpeza

Ao final da etapa de tratamento foi gerado um resumo do processo de limpeza dos dados. Resultados obtidos:

- Registros originais: 8453
- Registros duplicados removidos: 0
- Registros removidos por ausência de campos obrigatórios: 1722
- Registros removidos por restrições: 0
- Registros finais carregados: 6731

Os resultados indicam que a base original apresentou alto grau de consistência, não sendo identificados registros duplicados nem registros removidos por violação das restrições de domínio implementadas.

A principal redução observada ocorreu devido à ausência de atributos considerados obrigatórios para representação válida de partidas no modelo relacional.

4.6 Aplicação das restrições e integridade dos dados

Durante o processo de transformação e carga foram aplicadas restrições derivadas do modelo conceitual e do esquema relacional.

4.6.1 Restrições de domínio

Foram utilizadas regras de validação para garantir que valores inválidos não fossem inseridos no banco.

Entre as principais regras aplicadas:

- `ano_campeonato > 0`;
- `rodada > 0`;
- `publico ≥ 0`;
- `gols ≥ 0`;
- `colocacao` dentro do intervalo válido;
- estatísticas esportivas maiores ou iguais a zero.

Valores ausentes em atributos não obrigatórios foram mantidos como NULL para preservar o significado original da base.

No PostgreSQL, diferentemente do SQLite, as restrições de chave estrangeira são ativas por padrão, dispensando qualquer configuração explícita para habilitá-las.

4.6.2 Restrições de entidade

As entidades foram protegidas por chaves primárias e restrições de unicidade, garantindo que cada identificador representasse apenas um elemento do domínio.

4.6.3 Restrições de participação

As relações obrigatórias foram implementadas por meio de chaves estrangeiras obrigatórias e inserção controlada.

Assim, uma partida somente foi carregada quando existiam previamente registros válidos de campeonato, estádio e árbitro.

4.7 Estratégia de população do banco

A população do banco foi realizada em uma ordem específica para respeitar as dependências entre as tabelas e evitar violações de chave estrangeira.

Primeiro foram inseridas as entidades independentes, isto é, aquelas que não dependem de nenhuma outra tabela. Em seguida, foram inseridas as entidades dependentes, respeitando as relações definidas no modelo relacional.

A ordem para a inserção e sua justificativa são apresentados na Tabela 1.

Ordem	Tabela	Dependência
1	campeonato	nenhuma
2	pessoa	nenhuma
3	tecnico	pessoa
4	arbitro	pessoa
5	estadio	nenhuma
6	time	nenhuma
7	partida	campeonato, estadio, arbitro
8	estatistica	partida, time, tecnico

Tabela 1: Ordem de inserção das tabelas e suas dependências

Ordem	Tabela	Justificativa
1	campeonato	Não depende de nenhuma outra tabela
2	pessoa	Serve como superclasse para técnico e árbitro
3	tecnico	Depende de pessoa
4	arbitro	Depende de pessoa
5	estadio	Entidade independente usada por partida
6	time	Entidade independente usada por estatística
7	partida	Depende de campeonato, estadio e arbitro
8	estatistica	Depende de partida, time e técnico

Tabela 2: Justificativa para a ordem de inserção

A base original estava em formato tabular, com várias informações de uma partida na mesma linha. Durante a carga, cada linha do CSV foi decomposta em diferentes entidades.

Cada linha válida do CSV gerou um registro na tabela Partida e dois registros na tabela Estatística: um referente ao time mandante e outro referente ao time visitante.

4.8 Validação da carga

Após a execução do processo de ETL e população do banco de dados, foi realizada uma etapa de validação com o objetivo de verificar a consistência dos dados carregados e confirmar que as restrições definidas no modelo haviam sido respeitadas.

4.8.1 Quantidade de registros carregados

Após a conclusão da carga foram obtidas as seguintes quantidades por entidade:

- campeonato: 18 registros
- pessoa: 481 registros
- tecnico: 273 registros
- arbitro: 208 registros
- estadio: 82 registros
- time: 44 registros
- partida: 6731 registros
- estatistica: 13462 registros

Verificou-se que a tabela Estatística apresentou exatamente o dobro de registros da tabela Partida, confirmando a regra de transformação utilizada durante a carga, na qual cada partida gera dois registros estatísticos: um para o time mandante e outro para o visitante.

```
Resumo da limpeza:
Registros originais: 8453
Duplicatas removidas: 0
Removidos por campos obrigatórios: 1722
Removidos por restrições: 0
Registros finais: 6731
Banco criado com sucesso!
campeonato: 18 registros
pessoa: 481 registros
tecnico: 273 registros
arbitro: 208 registros
estadio: 82 registros
time: 44 registros
partida: 6731 registros
estatistica: 13462 registros
```

Figura 3: Resumo da limpeza e quantidade de registros carregados

4.8.2 Validação de domínio

Também foram realizadas consultas diretamente sobre o banco gerado para verificar possíveis violações das restrições de domínio.

Resultados:

- Ano inválido: 0
- Rodada inválida: 0
- Público negativo: 0
- Gols negativos: 0
- Estatísticas negativas: 0

Esses resultados indicam que nenhum registro persistido violou as regras estabelecidas no modelo conceitual.

```
Validação de domínio:  
Ano inválido: 0  
Rodada inválida: 0  
Público negativo: 0  
Gols negativos: 0  
Estatísticas negativas: 0
```

Figura 4: Resultado da validação das restrições de domínio

4.8.3 Validação de entidade

Foi executada a validação das restrições de entidade, verificando a unicidade das chaves primárias de todas as tabelas.

Resultados:

- campeonato.id_campeonato → 0 duplicados
- pessoa.id_pessoa → 0 duplicados
- tecnico.id_tecnico → 0 duplicados
- arbitro.id_arbitro → 0 duplicados
- estadio.id_estadio → 0 duplicados
- time.id_time → 0 duplicados
- partida.id_partida → 0 duplicados
- estatistica.id_estatistica → 0 duplicados

```
Validação de duplicatas:  
campeonato.id_campeonato: 0 duplicados  
pessoa.id_pessoa: 0 duplicados  
tecnico.id_tecnico: 0 duplicados  
arbitro.id_arbitro: 0 duplicados  
estadio.id_estadio: 0 duplicados  
time.id_time: 0 duplicados  
partida.id_partida: 0 duplicados  
estatistica.id_estatistica: 0 duplicados
```

Figura 5: Resultado da validação das restrições de entidade

4.8.4 Validação de participação

Por fim, foi realizada a validação das restrições de participação e integridade referencial.

Resultados:

- Partidas sem campeonato: 0
- Partidas sem estádio: 0
- Partidas sem árbitro: 0
- Estatísticas sem partida: 0
- Estatísticas sem time: 0

Com base nas validações executadas, concluiu-se que o banco resultante apresentou consistência estrutural, integridade referencial e aderência às restrições definidas nas etapas anteriores do projeto.

```
Validação de participação:
Partidas sem campeonato: 0
Partidas sem estádio: 0
Partidas sem árbitro: 0
Estatísticas sem partida: 0
Estatísticas sem time: 0
```

Figura 6: Resultado da validação de integridade referencial

4.9 Como executar

Para reproduzir a criação do banco de dados, é necessário ter instalados Docker e Python 3 no ambiente.

A estrutura esperada do projeto é:

```
Banco de Dados/
  dados/
    brasileirao.csv
  docker-compose.yml
  .env
  requirements.txt
  criar_banco.py
```

Primeiro, deve-se acessar a pasta do projeto e criar o ambiente virtual Python:

```
cd "Banco-de-Dados"
python -m venv .venv
source .venv/bin/activate
```

Em seguida, instalam-se todas as dependências:

```
pip install -r requirements.txt
```

Com o Docker em execução, sobe-se o contêiner PostgreSQL:

```
docker compose up -d
```

Por fim, executa-se o script de ETL:

```
python criar_banco.py
```

Ao final da execução, o banco de dados **brasileirao** estará disponível no PostgreSQL com todas as tabelas criadas, os dados carregados e as validações exibidas no terminal. Os dados persistem no volume Docker mesmo após reinicialização do contêiner.

5 Análise Exploratória de Dados

5.1 Visão Geral

A análise exploratória foi conduzida por meio de um Jupyter Notebook conectado diretamente ao banco de dados PostgreSQL, utilizando a biblioteca **psycopg2** para execução das consultas e as bibliotecas **Pandas**,

Matplotlib e Seaborn para construção dos gráficos.

Foram elaboradas quatro perguntas analíticas, cada uma explorando um aspecto diferente do campeonato. As consultas fazem uso de recursos variados da linguagem SQL, incluindo funções de agregação, junções, agrupamentos, expressões condicionais, CTEs e *window functions*.

5.2 Pergunta 1 — O Fator “Casa” por Estádio

5.2.1 Motivação

O fator casa é um dos fenômenos mais estudados no futebol. A pergunta investigada foi: *em quais estádios os times mandantes apresentam maior vantagem ofensiva em relação aos visitantes?*

5.2.2 Consulta SQL

A consulta agrupa as partidas por estádio, filtra apenas aquelas com público informado e exige ao menos 10 jogos para garantir amostra estatisticamente relevante.

```
SELECT
    e.nome_estadio,
    COUNT(p.id_partida) AS total_jogos,
    AVG(p.publico) AS media_publico,
    AVG(p.gols_mandante) AS media_gols_mandante,
    AVG(p.gols_visitante) AS media_gols_visitante
FROM partida p
JOIN estadio e ON p.id_estadio = e.id_estadio
WHERE p.publico IS NOT NULL
GROUP BY e.nome_estadio
HAVING COUNT(p.id_partida) >= 10
ORDER BY media_gols_mandante DESC
LIMIT 15;
```

Recursos SQL utilizados: SELECT, JOIN, WHERE, GROUP BY, HAVING, AVG, COUNT.

5.2.3 Visualização

O gráfico gerado é um gráfico de barras agrupadas, comparando a média de gols do mandante (verde) com a do visitante (vermelho) para os 15 estádios com maior média ofensiva dos mandantes.

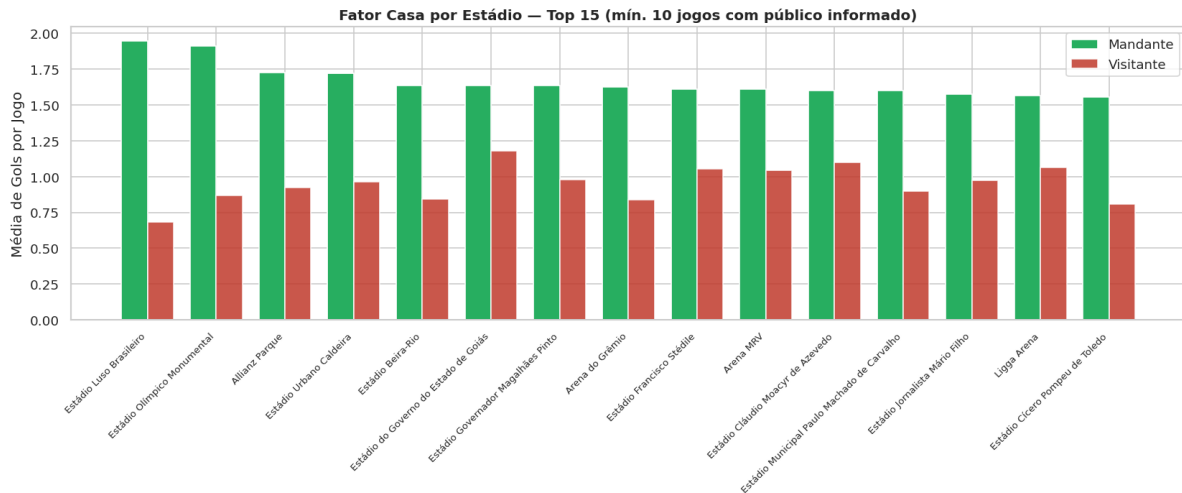


Figura 7: Fator casa por estádio — média de gols mandante vs. visitante (Top 15, mínimo 10 jogos)

5.2.4 Discussão

Estádios com maior público nem sempre concentram a maior diferença de gols entre mandantes e visitantes. Arenas menores e mais intimistas tendem a gerar pressão local mais intensa, o que pode beneficiar o mandante mesmo com menor capacidade de público. Estádios que aparecem no topo com baixa média de público são os casos mais interessantes: sugerem times que jogam poucos jogos em casa mas com alta intensidade, ou adversários que enfrentam dificuldades específicas naquele ambiente. Por outro lado, grandes arenas modernas com torcidas mistas podem “diluir” a pressão sobre o visitante, reduzindo a vantagem relativa do mandante.

5.3 Pergunta 2 — Eficiência Ofensiva vs. Valor do Elenco

5.3.1 Motivação

A pergunta investigada foi: *times com elencos mais caros são realmente mais eficientes, precisando de menos chutes para marcar um gol, ou a tática pode superar o valor de mercado?*

5.3.2 Consulta SQL

A consulta utiliza uma CTE para consolidar, por time, o valor médio do elenco, o total de chutes e o total de gols, calculando ao final a razão entre chutes e gols como métrica de eficiência ofensiva.

```
WITH EficienciaOfensiva AS (
  SELECT
    t.nome_time,
    AVG(est.valor_equipe_titular) AS valor_medio_elenco,
    SUM(est.chutes) AS total_chutes,
    SUM(CASE
      WHEN est.tipo_mando = 'mandante' THEN p.gols_mandante
      ELSE p.gols_visitante
    END) AS total_gols
  FROM estatistica est
  JOIN partida p ON est.id_partida = p.id_partida
  JOIN time t ON est.id_time = t.id_time
  GROUP BY t.nome_time
)
SELECT
  nome_time,
  ROUND(valor_medio_elenco::NUMERIC, 2) AS valor_medio_elenco,
  total_gols,
  total_chutes,
  ROUND((total_chutes::NUMERIC / NULLIF(total_gols, 0)), 2) AS chutes_por_gol
FROM EficienciaOfensiva
WHERE total_gols > 0
  AND valor_medio_elenco IS NOT NULL
  AND total_chutes IS NOT NULL
ORDER BY chutes_por_gol ASC;
```

Recursos SQL utilizados: CTE (WITH), JOIN, SUM, AVG, CASE, NULLIF, ROUND, cast explícito (::NUMERIC).

5.3.3 Visualização

O gráfico gerado é um gráfico de dispersão (*scatter plot*) com o valor médio do elenco no eixo *x* e a razão chutes por gol no eixo *y*. A coloração dos pontos segue um gradiente de vermelho (menos eficiente) a verde (mais eficiente), e todos os times são identificados com seus respectivos nomes. Uma linha de mediana tracejada é exibida como referência.

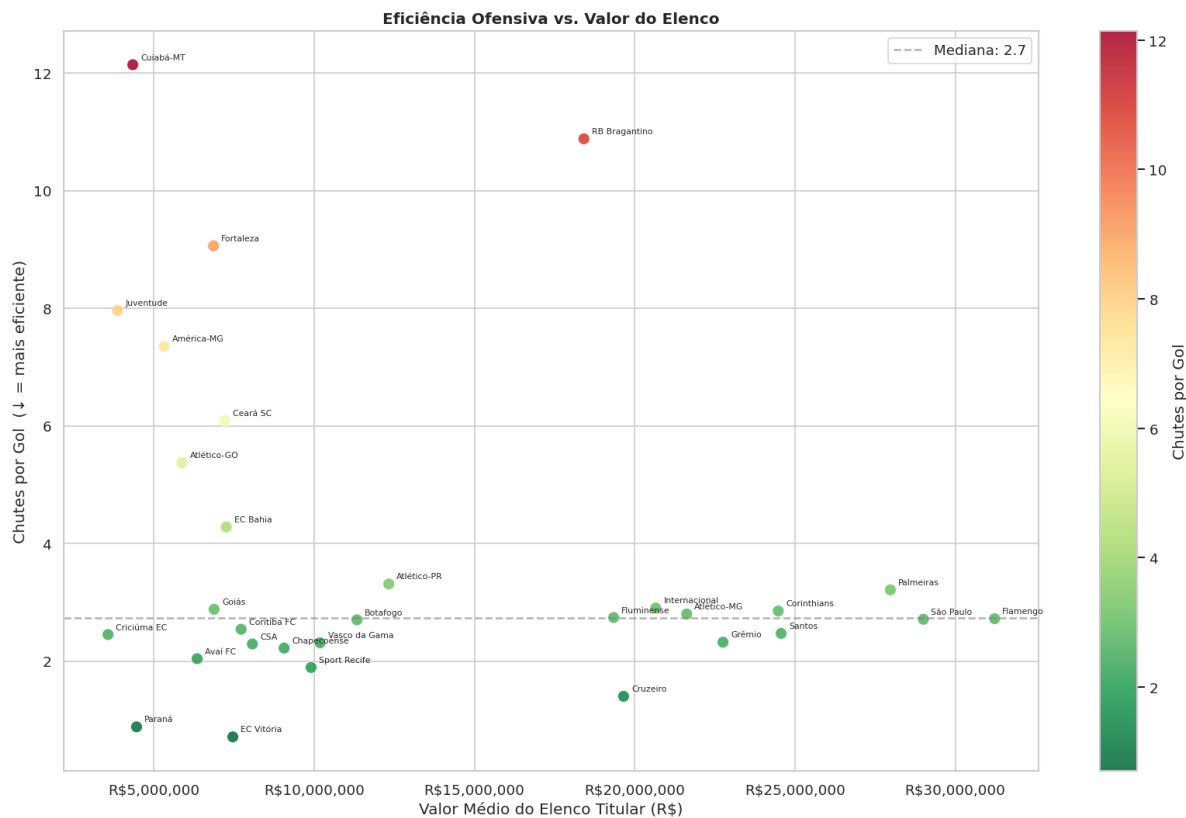


Figura 8: Eficiência ofensiva vs. valor do elenco — chutes por gol em função do valor médio do elenco titular

5.3.4 Discussão

A tendência esperada é que times com elencos mais caros precisem de menos chutes para marcar, refletindo maior qualidade técnica dos atletas. Um time posicionado no quadrante inferior esquerdo do gráfico (baixo valor de elenco, baixa razão chutes por gol) representa uma anomalia positiva: indica excelência tática ou um estilo de jogo reativo e eficiente, como o contra-ataque. O inverso também é revelador: elencos de alto valor com grande número de chutes por gol podem indicar domínio da posse de bola com baixa conversão, ou a presença de goleiros adversários em grande fase durante o período analisado. A dispersão dos pontos ao redor da mediana evidencia que o valor do elenco, por si só, não determina a eficiência ofensiva de forma linear.

5.4 Pergunta 3 — O Perfil dos Árbitros

5.4.1 Motivação

A pergunta investigada foi: *quais árbitros registram as maiores médias de faltas por partida, e como cada um se compara com a média geral do campeonato?*

5.4.2 Consulta SQL

A consulta une as tabelas de partida, árbitro e pessoa, e utiliza uma subconsulta para agregar o total de faltas por partida. Uma *window function* calcula a média geral do campeonato sobre os grupos já filtrados pelo HAVING.

```
SELECT
    pes.nome AS nome_arbitro,
    COUNT(DISTINCT p.id_partida) AS partidas_apitadas,
    ROUND(AVG(est_totais.total_faltas_partida)::NUMERIC, 2) AS media_faltas_arbitro,
```

```

        ROUND(
            (AVG(AVG(est_totais.total_faltas_partida)) OVER()))::NUMERIC, 2
        )
        AS media_geral_campeonato
FROM partida p
JOIN arbitro arb ON p.id_arbitro = arb.id_arbitro
JOIN pessoa pes ON arb.id_arbitro = pes.id_pessoa
JOIN (
    SELECT id_partida, SUM(faltas) AS total_faltas_partida
    FROM estatistica
    WHERE faltas IS NOT NULL
    GROUP BY id_partida
) est_totais ON p.id_partida = est_totais.id_partida
GROUP BY pes.nome
HAVING COUNT(DISTINCT p.id_partida) >= 5
ORDER BY media_faltas_arbitro DESC
LIMIT 20;

```

Recursos SQL utilizados: Window Function (OVER), subconsulta, COUNT(DISTINCT), AVG aninhado, JOIN múltiplo, HAVING.

5.4.3 Visualização

O gráfico gerado é um **gráfico de barras horizontal**, com os árbitros ordenados da maior para a menor média de faltas. As barras são coloridas em vermelho para árbitros acima da média geral e em azul para os abaixo. Uma linha vertical tracejada indica a média geral do campeonato.

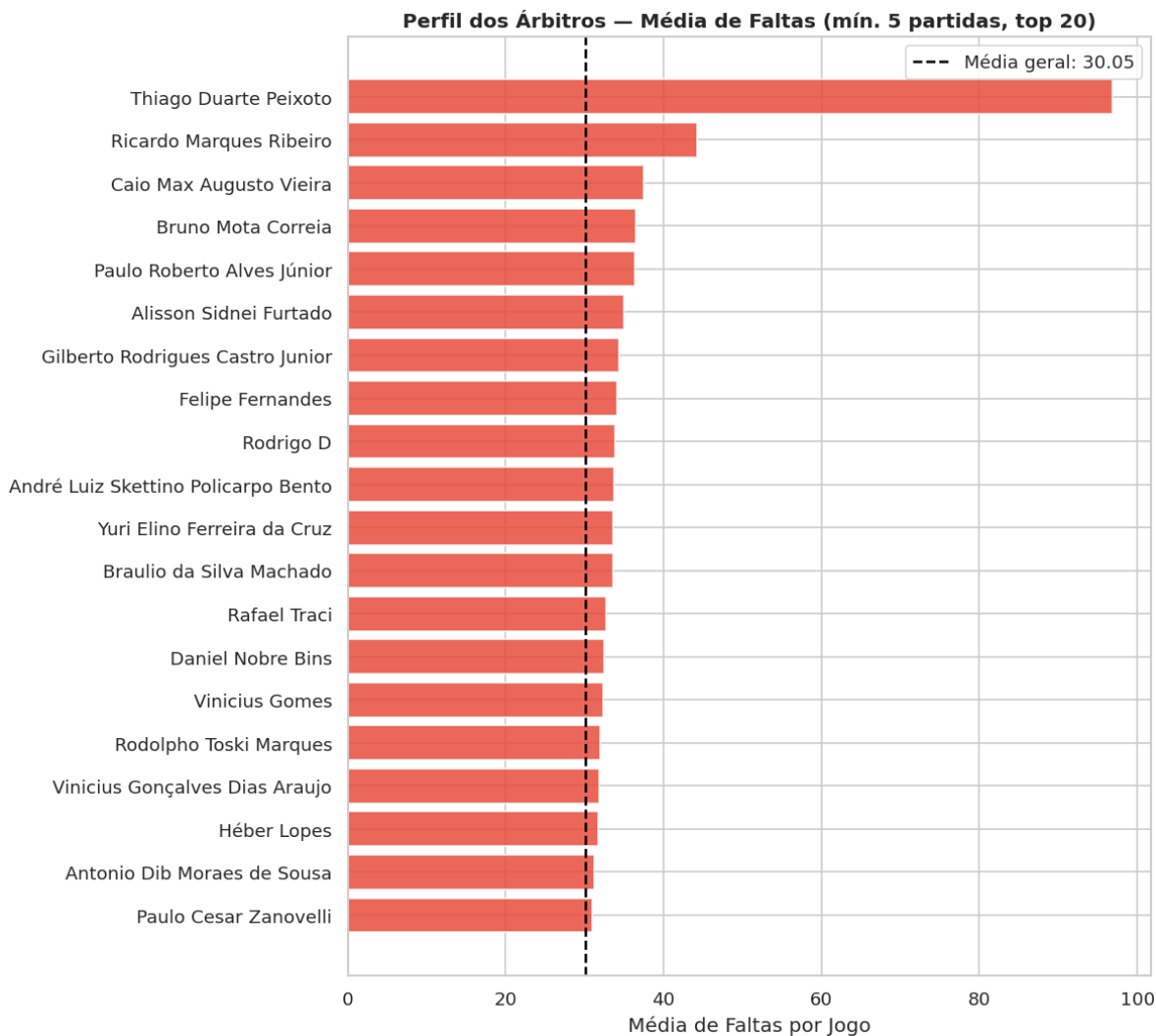


Figura 9: Perfil dos árbitros — média de faltas por partida vs. média geral do campeonato (mínimo 5 partidas, Top 20)

5.4.4 Discussão

Árbitros sistematicamente acima da média tendem a marcar praticamente qualquer contato físico, o que fragmenta o jogo e reduz o tempo efetivo de bola rolando. Esse comportamento pode ser uma característica individual ou refletir os times que aquele árbitro costuma apitar. Valores muito baixos de faltas merecem atenção especial do ponto de vista da qualidade dos dados: em anos mais antigos do campeonato, como 2005 a 2010, algumas súmulas foram preenchidas de forma incompleta na fonte original, podendo subestimar as médias de determinados árbitros. Uma análise complementar relevante seria cruzar a média de faltas com o número de cartões por partida para verificar se árbitros mais restritivos também punem mais.

5.5 Pergunta 4 — Juventude × Experiência na Tabela

5.5.1 Motivação

A pergunta investigada foi: *existe correlação entre a idade média do time titular e a colocação final no campeonato? Times mais experientes garantem posições melhores?*

5.5.2 Consulta SQL

A consulta agrega as estatísticas por colocação final, calculando a idade média geral, a menor e a maior idade média observada para times que alcançaram cada posição.

```
SELECT
    est.colocacao,
    ROUND(AVG(est.idade_media_titular)::NUMERIC, 2) AS idade_media_geral,
    MIN(est.idade_media_titular) AS time_mais_jovem,
    MAX(est.idade_media_titular) AS time_mais_velho
FROM estatistica est
WHERE est.colocacao IS NOT NULL
GROUP BY est.colocacao
ORDER BY est.colocacao ASC;
```

Recursos SQL utilizados: AVG, MIN, MAX, GROUP BY, WHERE, ORDER BY, ROUND.

5.5.3 Visualização

O gráfico gerado é um **gráfico de linha** com a idade média por colocação, acompanhado de uma banda sombreada representando o intervalo entre a menor e a maior idade média observada. Linhas de referência verticais tracejadas indicam o corte do G4 (posições 1–4) e o início da zona de rebaixamento (posição 17).

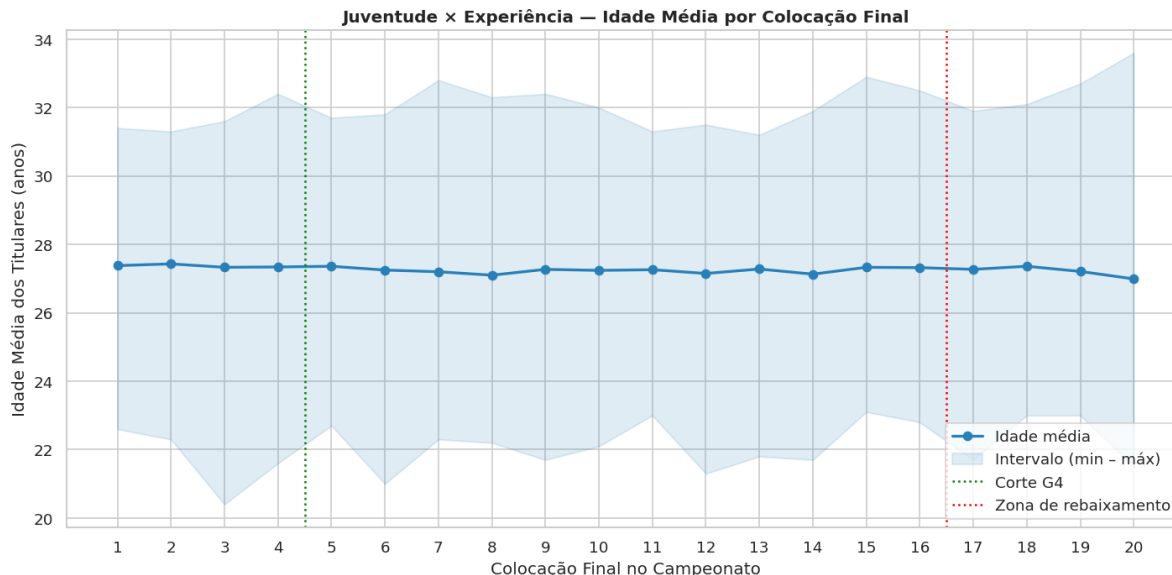


Figura 10: Juventude x experiência — idade média dos titulares por colocação final no campeonato

5.5.4 Discussão

Ao contrário da hipótese inicial de que a consistência ao longo de 38 rodadas favoreceria elencos visivelmente mais maduros no G4, a análise dos dados **refuta a correlação** entre a idade média do elenco titular e a colocação final. A linha de idade média se apresenta de forma praticamente constante (em torno de 27 anos) para todas as posições, do campeão ao último rebaixado. Isso indica que a proporção ideal entre juventude e experiência é um paradigma adotado homogeneamente por praticamente todos os clubes da Série A, não sendo um diferencial isolado de sucesso ou fracasso. A largura da banda sombreada é um indicador de variância: colocações com banda larga mostram que diferentes perfis etários (times excepcionalmente jovens ou mais velhos) foram capazes de atingir aquela posição ao longo das temporadas analisadas, reforçando a quebra do mito da experiência isolada.

5.6 Síntese e Conexão com Transparência de Dados

As quatro análises realizadas revelam padrões complementares sobre o Campeonato Brasileiro Série A, resumizados na Tabela 3.

Análise	Técnica SQL principal	Elemento gráfico
Fator casa por estádio	GROUP BY, HAVING, AVG	Barras agrupadas
Eficiência vs. valor	CTE, CASE, NULLIF	Dispersão (<i>scatter</i>)
Perfil dos árbitros	Window function, subconsulta	Barras horizontais
Juventude vs. experiência	MIN, MAX, AVG	Linha com banda

Tabela 3: Síntese das análises exploratórias realizadas

A disponibilização pública dos dados estatísticos do Brasileirão, conforme os princípios da **Lei de Acesso à Informação (LAI)**, viabiliza análises como as apresentadas neste trabalho. Sem acesso a dados estruturados, verificações como a da Pergunta 3 — que identifica padrões sistemáticos no comportamento de árbitros ao longo de temporadas — seriam inviáveis para pesquisadores independentes e veículos de imprensa. Esse tipo de auditoria orientada por dados promove a **transparência esportiva** e exemplifica como a abertura de informações de interesse público vai além do setor governamental, alcançando entidades que administram competições com amplo impacto social e econômico.

6 Conclusão

Este trabalho prático permitiu vivenciar o ciclo completo de vida de um projeto de banco de dados, partindo da modelagem conceitual até a extração de *insights* analíticos. A transformação dos dados brutos do Campeonato Brasileiro em um modelo relacional normalizado (3FN) estruturado em PostgreSQL garantiu a integridade e a consistência das informações.

A construção de um pipeline de ETL idempotente via Python evidenciou a importância do tratamento rigoroso de inconsistências antes de submeter os dados ao SGBD. A etapa de análise exploratória, por sua vez, demonstrou o valor prático de uma base de dados bem orquestrada, permitindo o uso de consultas SQL complexas para responder perguntas elaboradas sobre eficiência ofensiva, fator casa, perfil de arbitragem e impacto da idade no desempenho das equipes.

Por fim, o projeto comprovou que a aplicação correta dos conceitos de engenharia de dados, somada aos princípios de transparência e dados abertos (LAI), configura uma ferramenta fundamental para a auditoria, a geração de conhecimento e a interpretação crítica de cenários do mundo real.